

ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE
FAKULTA STAVEBNÍ, OBOR GEODÉZIE A KARTOGRAFIE
KATEDRA GEOMATIKY

Algoritmy digitální kartografie a GIS

Generalizace budov LOD0

Rok	Semestr	Skupina	Vypracovali	Datum
2025/2026	Letní		František Gurecký Adam Králič Vítek Veselý	12. 4. 2026

1 Zadání

Vstup: množina budov $B = \{B_i\}_{i=1}^n$, budova $B_i = \{P_{i,j}\}_{j=1}^m$

Výstup: $G(B_i)$

Ze souboru načtete vstupní data představovaná lomovými body budov a proveďte generalizaci budov do úrovně detailu LODO. Pro tyto účely použijte vhodnou datovou sadu, např. ZABAGED, testování proveďte nad třemi datovými sadami (historické centrum města, sídliště, izolovaná zástavba).

Pro každou budovu určete její hlavní směry metodami:

- Minimum Area Enclosing Rectangle,
- PCA.

U první metody použijte některý z algoritmů pro konstrukci konvexní obálky. Budovu při generalizaci do úrovně LODO nahraďte obdélníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy, jeho plocha bude stejná jako plocha budovy. Výsledky generalizace vhodně vizualizujte.

Otestujte a porovnejte efektivitu obou metod s využitím hodnotících kritérií. Pokuste se rozhodnout, pro které tvary budov dávají metody nevhodné výsledky, a pro které naopak poskytují vhodnou aproximaci.

2 Teoretická část

2.1 Konvexní obálka (Convex Hull)

Konvexní obálka slouží v naší aplikaci jako nezbytný přípravný krok pro zefektivnění a umožnění výpočtu dvou hlavních metod:

- **Metoda MAER (Minimum Area Enclosing Rectangle):** Algoritmus vychází z teoretického předpokladu, že alespoň jedna strana minimálního ohraničujícího obdélníku musí být rovnoběžná s hranou konvexní obálky. Výpočet nad konvexní obálkou výrazně redukuje počet testovaných směrů (hran) oproti původnímu polygonu.
- **Metoda Weighted Bisector:** Tato metoda využívá vrcholy konvexní obálky k nalezení dvou nejdelších úhlopříček (diagonál) objektu. Právě body tvořící konvexní obálku reprezentují extrémní body budovy, které definují její celkový rozsah a hlavní směr.

2.1.1 Implementované algoritmy

Graham Scan Tento algoritmus využívá zásobníkovou techniku a třídění bodů. Implementace probíhá v následujících krocích:

1. **Nalezení pivotu:** Jako pivot je vybrán bod s největší souřadnicí y (nejjižnější bod).
2. **Vytvoření hvězdicového polygonu:** Ostatní body jsou seřazeny podle úhlu, který svírají s pomocným bodem a pivotem. V implementaci je využita funkce `get2LinesAngle` pro přesné seřazení.
3. **Grahamův cyklus:** Pomocí zásobníku (`deque`) procházíme seřazené body. Pokud přidání nového bodu vytváří pravotočivou zatačku (kontrolováno funkcí `analyzePointAndLineRelation`), je předchozí bod ze zásobníku odstraněn, dokud není dosaženo konvexního stavu.

Jarvisův pochod Jedná se o metodu „balení dárků“ (gift wrapping), která je v kódu implementována ve verzi optimalizované pro potřeby kartografické generalizace:

1. Nalezne se počáteční pivot q (minimální y) a nejlevější bod s pro definici počátečního směru.
2. Algoritmus iterativně hledá bod, který svírá největší úhel vzhledem k aktuální hraně.
3. Proces končí, jakmile se algoritmus vrátí k počátečnímu pivotu q .

2.1.2 Metoda zametačské přímky (Sweeping Line)

- Nejdříve body setřídí podle souřadnice x (a následně y).
- Postupně přidává body a udržuje spojový seznam (`next/prev` ukazatele), který definuje hranici obálky.
- Efektivně ošetřuje horní a dolní větev obálky, přičemž průběžně odstraňuje nekonvexní vrcholy.

2.2 Algoritmy pro určení hlavního směru budovy

Pro generalizaci budov do úrovně detailu LOD0 je stěžejní správně určit jejich dominantní orientaci (hlavní směr). V rámci třídy `Algorithms` bylo implementováno pět nezávislých metod.

2.2.1 Metoda minimálního ohraničujícího obdélníku (MAER)

Tato metoda vyhledává ohraničující obdélník (Minimum Bounding Rectangle) s absolutně nejmenší možnou plochou. Využívá se geometrického předpokladu, že alespoň jedna strana tohoto obdélníku musí být rovnoběžná s některou hranou konvexní obálky polygonu.

- **Postup výpočtu:** Algoritmus (`createMAER`) nejprve zkonstruuje konvexní obálku budovy. Následně iteruje přes všechny její hrany a pro každou počítá její směrnik $\sigma = \text{atan2}(\Delta y, \Delta x)$.
- Polygon je následně dočasně zrotován o úhel $-\sigma$ (zarovnání s osou X) a je vypočten osově paralelní hraniční box (`minMaxBox`).
- Směr, který generuje obdélník s nejmenší plochou, je vybrán jako hlavní směr natočení budovy.

2.2.2 Analýza hlavních komponent (PCA)

Statistická metoda, která analyzuje rozptyl lomových bodů budovy. Hlavní směry budovy odpovídají směrům maximální variability dat.

- **Postup výpočtu:** Ze souřadnic vrcholů polygonu je sestavena matice A , ze které je pomocí knihovny NumPy vypočtena kovarianční matice C .
- Nad maticí C je proveden rozklad na singulární hodnoty (SVD – Singular Value Decomposition), čímž získáme matice U, Σ, V .
- **Určení úhlu:** Úhel natočení σ je extrahován z prvků prvního vlastního vektoru matice V pomocí funkce:

$$\sigma = \text{atan2}(V_{0,1}, V_{0,0}) \quad (1)$$

2.2.3 Metoda nejdelší hrany (Longest Edge)

Geometricky nejjednodušší metoda (`simplifyBuildingLongestEdge`), která předpokládá, že nejdelší stěna budovy nejlépe definuje její celkovou orientaci.

- **Postup výpočtu:** Algoritmus prochází všechny segmenty polygonu a počítá jejich Euklidovskou vzdálenost.
- U segmentu s maximální délkou je vypočten jeho směrnik. O tento úhel je následně orientován výsledný generalizovaný obdélník.

2.2.4 Vážený průměr stěn (Wall Average)

Tato metoda (`simplifyBuildingWallAverage`) zohledňuje směry všech stěn budovy, přičemž delší stěny mají na výsledný směr větší vliv.

- **Postup výpočtu:** Pro každou hranu je vypočten její směrnik σ_i . Následně se počítá vnitřní úhel ω mezi sousedními hranami a jeho odchylka r od pravého úhlu (zbytek po dělení $\frac{\pi}{2}$).
- **Vážený průměr:** Celkové natočení budovy σ_{rot} je dáno přičtením váženého průměru odchylek k počátečnímu směru σ_0 , kde váhou s je délka příslušné hrany:

$$\sigma_{rot} = \sigma_0 + \frac{\sum (r_i \cdot s_i)}{\sum s_i} \quad (2)$$

2.2.5 Vážená osa (Weighted Bisector)

Metoda (`simplifyBuildingWeightedBisector`) využívající vnitřní úhlopříčky k nalezení podélné osy objektu.

- **Postup výpočtu:** Pro zefektivnění výpočtu jsou uvažovány pouze vrcholy konvexní obálky. Algoritmus otestuje všechny dvojice bodů a nalezne dvě nejdelší diagonály s délkami s_1, s_2 a jejich směrníky σ_1, σ_2 .

- **Výsledný směr:** Hlavní orientace je vypočtena jako vážený průměr směrnic těchto dvou úhlopříček:

$$\sigma = \frac{s_1\sigma_1 + s_2\sigma_2}{s_1 + s_2} \quad (3)$$

2.3 Zachování plochy budovy

Klíčovým požadavkem při generalizaci budov do úrovně detailu LOD0 je zachování původní rozlohy objektu. Poté, co je určen hlavní směr budovy a vytvořen ohraničující obdélník (MBR), je nutné jeho rozměry upravit tak, aby se jeho plocha přesně rovnala ploše původního polygonu. V třídě `Algorithms` je tento proces rozdělen do dvou hlavních kroků.

2.3.1 Výpočet plochy polygonu

Pro výpočet plochy libovolného n -úhelníku (původní budovy i vypočteného obdélníku) je v metodě `getPolygonArea` implementován L'Huilierův vzorec. Algoritmus iteruje přes všechny vrcholy polygonu a vypočítá plochu P pomocí následujícího vztahu:

$$P = \frac{1}{2} \left| \sum_{i=0}^{n-1} x_i \cdot (y_{i+1} - y_{i-1}) \right| \quad (4)$$

Kde indexy jsou uvažovány v modulu n (tedy pro první vrchol se jako předchozí bere poslední vrchol polygonu a naopak). Použití absolutní hodnoty zajišťuje správný výsledek nezávisle na tom, zda jsou vrcholy zadány po směru nebo proti směru hodinových ručiček.

2.3.2 Škálování ohraničujícího obdélníku

Jakmile známe plochu původní budovy (P_{build}) a plochu jejího minimálního ohraničujícího obdélníku (P_{rect}), můžeme provést změnu jeho velikosti v metodě `resizeRectangle`. Postup je následující:

1. **Výpočet plošného koeficientu:** Vypočítá se poměr ploch $k = \frac{P_{build}}{P_{rect}}$. Protože se plocha mění s druhou mocninou délkových rozměrů, pro změnu délek stran (a souřadnic) se musí použít odmocnina z tohoto koeficientu, tedy $\sqrt{k}$.
2. **Určení těžiště obdélníku:** Aby obdélník zůstal na svém místě a nedeformovala se jeho poloha vůči původní budově, škálování probíhá vůči jeho středu (těžišti). Střed $C = (x_c, y_c)$ je vypočten jako prostý průměr souřadnic jeho čtyř vrcholů:

$$x_c = \frac{1}{4} \sum_{i=0}^3 x_i, \quad y_c = \frac{1}{4} \sum_{i=0}^3 y_i \quad (5)$$

3. **Přepočet souřadnic vrcholů:** Každý vrchol obdélníku je následně posunut směrem k těžišti (nebo od něj) vynásobením jeho vzdálenosti od těžiště faktorem $\sqrt{k}$. Nové souřadnice x'_i, y'_i se získají transformací:

$$x'_i = (x_i - x_c) \cdot \sqrt{k} + x_c \quad (6)$$

$$y'_i = (y_i - y_c) \cdot \sqrt{k} + y_c \quad (7)$$

Výsledkem této transformace je zjednodušený obdélník, který má správnou orientaci odvozenou z jednoho z předchozích algoritmů, jehož střed leží v těžišti původního ohraničujícího boxu a jehož plošná výměra přesně odpovídá původní budově.

3 Architektura aplikace a implementace

Hlavní komponenty systému jsou rozděleny do čtyř základních modulů.

3.1 Třída Algorithms (algorithms.py)

Tato třída představuje výpočetní jádro aplikace a neobsahuje žádné prvky uživatelského rozhraní. Soustředí se výhradně na matematické a geometrické operace.

- **Základní geometrie:** Obsahuje funkce pro výpočet vzdáleností (`get2PointDistance`), úhlů (`get2LinesAngle`) a vektorových součinů (`crossProduct`) pro určení vzájemné polohy bodu a přímky.
- **Konvexní obálky:** Implementuje různé přístupy ke konstrukci konvexní obálky, konkrétně Graham Scan, Jarvisův pochod a metodu zametačské přímky (*Sweeping Line*).
- **Generalizační metody:** Obsahuje implementace všech pěti metod pro určení hlavního směru budovy (MAER, PCA, Wall Average, Longest Edge a Weighted Bisector) a funkce pro následné škálování obdélníků na požadovanou plochu.
- **Statistický modul:** Funkce `getStatisticsSummary` provádí závěrečné vyhodnocení úspěšnosti generalizace na základě úhlových odchylek hran.

3.2 Třída Draw (draw.py)

Modul zajišťuje veškerou interakci s grafickým plátnem a správu vizualizace dat.

- **Správa událostí:** Obsahuje handlersy pro události myši (`wheelEvent` pro zoom, `mousePressEvent` pro kreslení) a klávesnice (`keyPressEvent` pro posun plátna – pan).
- **Transformace souřadnic:** Realizuje přepočty mezi reálnými souřadnicemi (např. S-JTSK) a souřadnicemi obrazovky, včetně inverze osy Y a implementace funkcí pro automatické centrování dat (`zoomToData`).
- **Optimalizace vykreslování:** Pro plynulý chod aplikace při práci s velkými datovými sady je využito cacheování pomocí objektu `QPixmap`. Překreslování všech polygonů probíhá pouze v případě změny dat nebo měřítko (příznak `cache_dirty`), jinak je na obrazovku pouze vykreslován hotový bitmapový obraz.

3.3 Třída MainForm (MainForm.py)

Tato třída definuje hlavní okno aplikace a zajišťuje orchestraci celého programu.

- **Uživatelské rozhraní:** Definuje rozvržení prvků, jako je menu, Toolbar a logovací okno (QPlainTextEdit).
- **Propojení logiky:** Využívá mechanismus signálů a slotů k propojení uživatelských akcí (např. kliknutí na tlačítko PCA) s příslušnými výpočetními metodami v třídě `Algorithms` a následnému zobrazení výsledků v třídě `Draw`.

3.4 Třída Polygon (polygon.py)

Vlastní datová struktura, která rozšiřuje standardní třídu `QPolygonF` z knihovny Qt.

- **Metadata:** Ke standardním souřadnicím přidává atributy jako `id`, `bbox` (hraniční box), `simplified_pol` (odkaz na vypočtenou zjednodušenou geometrii) a `classified` (příznak, zda budova vyhověla statistickým kritériím).
- **Metody:** Nabízí pomocné funkce pro snadné přidávání vrcholů nebo import dat z jiných instancí polygonů.

4 Uživatelské rozhraní a manuál

Aplikace je navržena s využitím grafického frameworku PyQt6, což zajišťuje přehlednost a intuitivní ovládání. Uživatelské rozhraní se skládá z několika hlavních prvků, které umožňují interaktivní práci s prostorovými daty.

4.1 Popis hlavního okna

Hlavní okno aplikace (`MainForm`) je vertikálně rozděleno pomocí prvku `QSplitter` na dvě hlavní části:

- **Grafické plátno (Canvas):** Větší, horní část okna slouží pro samotnou vizualizaci prostorových dat a výsledků generalizace. Je zprostředkováno vlastní třídou `Draw`, která dědí z `QWidget`.
- **Logovací panel (Log):** Spodní textová oblast (`QPlainTextEdit`) slouží k výpisu stavových informací, systémového času jednotlivých operací a především pro zobrazení konečného statistického vyhodnocení úspěšnosti generalizace.

Prostorové rozložení těchto dvou prvků si může uživatel interaktivně měnit tažením za dělicí příčku.

4.2 Základní ovládání a interakce

Plátno aplikace (Canvas) podporuje plynulý pohyb a vlastní tvorbu dat díky implementaci událostí myši a klávesnice v modulu `draw.py`:

- **Interaktivní kreslení:** Uživatel může klikáním levým tlačítkem myši do plochy plátna manuálně zadávat vrcholy vlastního polygonu (`mousePressEvent`). Každým kliknutím se přidá nový bod do aktuálně vytvářeného tvaru.
- **Posun pohledu (Pan):** Pohyb nad mapou je řešen pomocí směrových šipek na klávesnici (`keyPressEvent`). Každý stisk posune kameru o definovaný krok v daném směru.
- **Přiblížení a oddálení (Zoom):** Změna měřítka se provádí pomocí kolečka myši (`wheelEvent`). Přibližování přitom respektuje aktuální polohu kurzoru na obrazovce, což zajišťuje přirozenou navigaci v datech.

4.3 Funkce horního menu a nástrojové lišty

V horní části aplikace se nachází hlavní menu a nástrojová lišta (Toolbar) sdružující ikony pro rychlý přístup ke všem klíčovým funkcím programu:

1. Práce s daty (File)

- *Open:* Otevře standardní systémový dialog pro výběr vstupního souboru ve formátu ESRI Shapefile (*.shp). Po načtení aplikace automaticky přizpůsobí přiblížení tak, aby byla viditelná všechna data (`zoomToData`).
- *Exit:* Ukončí aplikaci.

2. Metody generalizace (Simplify)

- Rozbalovací nabídka a příslušná tlačítka v liště spouští jednotlivé algoritmy nad načtenými daty: *Minimal Bounding Rectangle (MAER)*, *PCA*, *Weighted Bisector*, *Wall Average* a *Longest Edge*.
- Po kliknutí na vybranou metodu se spustí výpočet, plátno se překreslí novými obdélníky a do logovacího okna se vypíše podrobná statistika úspěšnosti.

3. Zobrazení a čištění (View)

- *Clear Results:* Smaže z plátna vypočtené ohraničující obdélníky (výsledky), ale ponechá zobrazené původní budovy.
- *Clear All:* Kompletně vymaže celou paměť plátna, odstraní původní data i výsledky a připraví plochu pro novou práci.

5 Načítání dat a práce se souřadnicemi

Tato kapitola popisuje proces importu geometrických dat a jejich následnou transformaci z reálných souřadnicových systémů do souřadnicového systému zobrazovacího zařízení (plátna aplikace).

5.1 Zpracování formátu Shapefile (.shp)

Pro import prostorových dat byl zvolen formát ESRI Shapefile, který je průmyslovým standardem v oblasti GIS. Načítání je realizováno v metodě `handleFileOpen` a následně zpracováno funkcí `saveSHPData` s využitím knihovny `pyshp`.

Proces importu probíhá v následujících krocích:

- **Otevření souboru:** Pomocí standardního dialogu `QFileDialog` uživatel vybere soubor typu `.shp`.
- **Iterace přes prvky:** Knihovna `shapefile.Reader` umožňuje iterovat přes jednotlivé tvary (*shapes*) v datové sadě.
- **Extrakce lomových bodů:** Pro každý polygon jsou extrahovány dvojice souřadnic (x, y) z pole `shape.points`. Tyto body jsou následně transformovány a uloženy jako instance třídy `Polygon` (potomek `QPolygonF`).
- **Načtení Bounding Boxu:** Z metadat každého tvaru je načten jeho obdélníkový rozsah (*bbox*), který slouží pro rychlé výpočty viditelnosti a automatické centrování dat.

5.2 Transformace souřadnicových systémů

Vzhledem k tomu, že vstupní data (např. v systému S-JTSK) a grafické rozhraní (Qt6 Canvas) používají odlišné konvence, je nutné provádět transformace souřadnic.

5.2.1 Posuny a offsety

Souřadnice v systémech jako S-JTSK dosahují vysokých hodnot (např. $x \approx 675\,000$, $y \approx 1\,100\,000$). Pro zachování numerické stability a snazší manipulaci v rámci plátna je při načítání aplikován konstantní *offset*:

$$x_{off} = x_{raw} + offset_x \quad (8)$$

Tento posun přibližuje data k počátku souřadnicového systému aplikace.

5.2.2 Inverze osy Y

Zatímco v kartografických systémech osa y roste směrem nahoru (na sever), v počítačové grafice (Qt) roste osa y směrem dolů. V kódu je tato transformace ošetřena následovně:

$$y_{canvas} = -(y_{raw} + offset_y) \quad (9)$$

Tato operace zajistí, že severní orientace budov zůstane zachována i po vykreslení na monitor.

5.2.3 Dynamické zobrazení (Zoom a Pan)

Pro interaktivní prohlížení dat je implementována matice transformace `QTransform`, která kombinuje aktuální úroveň přiblížení (*Zoom*) a posun pohledu (*Pan*):

- **Zoom:** Škálování souřadnic koeficientem `__zoom`, který se mění pomocí kolečka myši [?].
- **Pan:** Přičtení vektoru `__pan` k souřadnicím, umožňující pohyb po mapě šipkami nebo myší.

5.2.4 Automatické centrování (Zoom to Data)

Metoda `zoomToData` vypočítá globální *Bounding Box* všech načtených polygonů. Na základě šířky a výšky okna aplikace následně dopočítá optimální `__zoom` a `__pan` tak, aby byla celá datová sada viditelná s definovaným okrajem (*padding*).

6 Hodnocení úspěšnosti a statistika

Kvantitativní vyhodnocení kvality generalizace je nezbytné pro porovnání efektivity jednotlivých algoritmů. V aplikaci je tento proces realizován metodou `getStatisticsSummary` ve třídě `Algorithms`.

6.1 Metodika hodnocení

Základním principem hodnocení je porovnání směru každé stěny původního polygonu se směrem hran výsledného obdélníku. Vzhledem k tomu, že obdélník reprezentuje čtyři směry navzájem otočené o 90° ($\pi/2$), výpočet pracuje se zbytkovými úhly (rezidui) po dělení touto hodnotou.

Algoritmus nejprve určí hlavní směr obdélníku σ_{MBR} a vypočítá jeho zbytek r po dělení pravým úhlem. Následně pro každou z n hran původní budovy vypočítá její směr σ_i a příslušný zbytek r_i . Cílem je minimalizovat rozdíl mezi těmito zbytky.

6.2 Statistické ukazatele

Pro souhrnné vyhodnocení jsou počítány dva klíčové ukazatele úhlové odchylky:

1. **Průměrná úhlová odchylka ($\Delta\sigma_1$):** Charakterizuje systematické pootočení výsledku.

$$\Delta\sigma_1 = \frac{\pi}{2n} \sum_{i=1}^n (r_i - r) \quad (10)$$

2. **Kvadratická úhlová odchylka ($\Delta\sigma_2$):** Reprezentuje celkovou míru "nonlinearity" a nepravidelnosti budovy vůči zvolenému směru.

$$\Delta\sigma_2 = \frac{\pi}{2n} \sqrt{\sum_{i=1}^n (r_i - r)^2} \quad (11)$$

Při výpočtu rozdílu ($r_i - r$) je aplikována korekce, aby výsledek vždy ležel v intervalu $(-\pi/4, \pi/4]$, což ošetřuje případy na hranici kvadrantů.

6.3 Klasifikace výsledků

Na základě vypočtených odchylek je každá budova klasifikována jako úspěšně nebo neúspěšně generalizovaná. V aplikaci je nastavena tolerance $\epsilon = 10^\circ$ (převáděno na radiány jako $10 \cdot \pi/180$). Budova je označena jako nevyhovující (`classified = False`), pokud platí:

$$|\Delta\sigma_1| > \epsilon \quad \text{nebo} \quad \Delta\sigma_2 > \epsilon \quad (12)$$

Tento přístup umožňuje automaticky identifikovat objekty, u nichž zvolená metoda generalizace selhala (např. u budov s velmi nepravidelným tvarem nebo u kruhových objektů).

6.4 Vizuální interpretace

Výsledky statistického hodnocení jsou přímo propojeny s vykreslovacím modulem v souboru `draw.py`. Pro rychlou vizuální kontrolu jsou zjednodušené polygony vykreslovány s odlišným stylem:

- **Modrá:** Budovy splňující toleranci. Obdélník dobře vystihuje hlavní směr objektu.
- **Červená barva:** Budovy překračující toleranci. Tyto případy vyžadují manuální revizi nebo použití jiného typu generalizace (např. nahrazení jiným tvarem než obdélníkem).

Souhrnné statistiky za celou datovou sadu (procentuální podíl správně generalizovaných budov) jsou po dokončení výpočtu vypsány do logovacího okna aplikace.

7 Závěr

V Praze dne 12. 4. 2026

František Gurecký
Adam Králič
Vítek Veselý